# Tutorial on the MPL interface to MPI

Victor Eijkhout

Morehouse 2026

# Justification

While the C API to MPI is usable from C++, it feels very unidiomatic for that language. Message Passing Library (MPL) is a modern C++17 interface to MPI. It is both idiomatic and elegant, simplifying many calling sequences.

`https://github.com/rabauke/mpl`

# 1. Basics

# Environment

Software environment setup

```
module load mpl
```

which defines `TACC_MPL_INC`, `TACC_MPL_DIR`

# Header file

To compile MPL programs, add a line

```
#include <mpl/mpl.hpp>
```

to your file. You need to add a path to your compile line:

```
mpicxx -o mpiprog -I${MPL_LOCATION}/include mympiprog.cpp
```

where *MPL_LOCATION* is system-dependent.

In CMake, MPL can be found:

```
find_package( mpl )
// this defines the target: mpl::mpl
```

# Init, finalize

There is no initialization or finalize call.

*Implementation note: Initialization is done at the first* `mpl::environment` *method call, such as* `comm_world`*.*

# World communicator

The naive way of declaring a communicator would be:

```
// commrank.cxx
mpl::communicator comm_world =
  mpl::environment::comm_world();
```

calling the predefined environment method *comm_world*.

However, if the variable will always correspond to the world communicator, it is better to make it *const* and declare it to be a reference:

```
const mpl::communicator &comm_world =
  mpl::environment::comm_world();
```

Most MPI functions will be methods of a communicator object.

# Processor name

The processor name is a method of the environment class:

Code:

```
// procname.cxx
procno = comm_world.rank();
string procname =
  mpl::environment::
      processor_name();
std::println
  ( "[{}] running on: {}",
    procno,procname );
```

Output:

```
[5] running on:
    ↪c455-092.stampede3.tacc.ute
[4] running on:
    ↪c455-091.stampede3.tacc.ute
[2] running on:
    ↪c454-024.stampede3.tacc.ute
[3] running on:
    ↪c454-024.stampede3.tacc.ute
[1] running on:
    ↪c454-023.stampede3.tacc.ute
[0] running on:
    ↪c454-023.stampede3.tacc.ute
```

(One of the few functions not associated with a communicator.)

# Rank and size

The rank of a process and the size of a communicator are both methods of the *communicator* class:

```
const mpl::communicator &comm_world =
    mpl::environment::comm_world();
int procid = comm_world.rank();
int nprocs = comm_world.size();
```

# Exercise 1 `commrank`

1. Use the rank/size functions to let each process print out

   *Hello from process* 3 *out of* 5

2. Let just one process print out

   *Hello from* 5 *processes*

# Predefined communicators

The *environment* namespace has the equivalents of *MPI_COMM_WORLD* and *MPI_COMM_SELF*:

```
const communicator& mpl::environment::comm_world();
const communicator& mpl::environment::comm_self();
```

Uses of *MPI_COMM_NULL* are handled differently.

# Communicator duplication

Communicators can be duplicated but only during initialization. Copy assignment has been deleted. Thus:

```
// CORRECT:
mpl::communicator init = comm;
// WRONG:
mpl::communicator init;
init = comm;
```

# Communicator passing

Pass communicators by reference to avoid communicator duplication:

```
// commpass.cxx
// BAD! this does a MPI_Comm_dup.
void comm_val( const mpl::communicator comm );

// correct!
void comm_ref( const mpl::communicator &comm );
```

# Communicator copying

The communicator class has its copy operator deleted; however, copy initialization exists:

```
// commcompare.cxx
const mpl::communicator &comm =
  mpl::environment::comm_world();
println( "same: {}",(comm==comm) );

mpl::communicator copy =
  mpl::environment::comm_world();
println( "copy: {}",(comm==copy) );

mpl::communicator init = comm;
println( "init: {}",(comm==init) );
```

(This outputs true/false/false respectively.)

*Implementation note: The copy initializer performs an* `MPI_Comm_dup`.

# 2. Collectives

# Introduction

Collectives have many polymorphic variants, for instance for ‘in place’, and buffer handling.

Operators are handled through functors.

# Scalar buffers

*Buffer* type handling is done through polymorphism (templating and ADL):
no explicit indication of types.

Scalars are handled as such:

```
float x,y;
comm.bcast( 0,x ); // note: root first
```

# Vector buffers

If your buffer is a *std::vector* you need to take the *.data()* component of it and explicitly indicate a 'layout'.

The simplest layout is a *contiguous_layout*:

```
vector<float> xx(2),yy(2);
comm.allreduce( mpl::plus<float>(),
    xx.data(), yy.data(), mpl::contiguous_layout<float>(2) );
```

More about layouts later; see note 64 and on.

For now, interpret it as a way of indicating the count/type part of a buffer specification.

# Array buffers

You can pass a C-style 'automatic array' as buffer, requiring a layout:

```
// vector of 50 floats
vector<float> ar(50);
auto root = 0;
auto data = ar.data(); // or &(ar[0]) or &ar.front()
auto layout = mpl::contiguous_layout<float>(50)
comm_world::bcast( root,data,layout );
```

# Reduction operator

Versions with separate send/receive buffers, and reduction in place:

```
// separate recv buffer
comm_world.allreduce(mpl::plus<float>(), proc_data,reduce_data);
// in place
comm_world.allreduce(mpl::plus<float>(), proc_data);
```

Note the parentheses after the operator. Also note that the operator comes first, not last as in the C Application Programmer Interface (API).

Available: `max`, `min`, `plus`, `multiplies`, `logical_and`, `logical_or`, `logical_xor`, `bit_and`, `bit_or`, `bit_xor`.

*Implementation note: The reduction operator has to be compatible with* `T(T,T)>`.

# Reference: MPI_Allreduce

```
template<typename T , typename F >
void mpl::communicator::allreduce
   ( F, const T &, T & ) const;
   ( F, const T *, T *,
     const contiguous_layout< T > & ) const;
   ( F, T & ) const;
   ( F, T *, const contiguous_layout< T > & ) const;
F : reduction function
T : type
```

# Reference: MPI_Reduce

```
void mpl::communicator::reduce
    // root, in place
    ( F  f,int  root_rank,T &  sendrecvdata ) const
    ( F  f,int  root_rank,T *  sendrecvdata,const contiguous_layout< T > &  l )
      const
    // non-root
    ( F  f,int  root_rank,const T &  senddata ) const
    ( F  f,int  root_rank,
            const T * senddata,const contiguous_layout< T > &  l ) const
    // general
    ( F  f,int  root_rank,const T &  senddata,T &  recvdata ) const
    ( F  f,int  root_rank,
            const T *  senddata,T *  recvdata,const contiguous_layout< T > &  l )
      const
```

# Exercise 2 `reduce`

1. Let each process have a random value.
2. Normalize this number by the sum of the values.
3. For bonus points sum the normalized values and check that they sum to one.

# Broadcast

The broadcast call comes in two variants, with scalar argument and general layout:

```
 template<typename T >
void mpl::communicator::bcast
   ( int root_rank, T &data ) const;
void mpl::communicator::bcast
   ( int root_rank, T *data, const layout< T > &l ) const;
```

(The root argument comes first in a departure from the C API.)

# Reference: MPI_Bcast

```
template<typename T >
void mpl::communicator::bcast
   ( int  root, T &  data ) const
   ( int  root, T *  data, const layout< T > &  l ) const
```

# Gather scatter

Gathering (by *communicator::gather*)
or scattering (by *communicator::scatter*)
a single scalar takes a scalar argument and a raw array:

```
vector<float> v;
float x;
comm_world.scatter(0, v.data(), x);
```

If more than a single scalar is gathered, or scattered into, it becomes necessary to specify a layout:

```
vector<float> vrecv(2),vsend(2*nprocs);
mpl::contiguous_layout<float> twonums(2);
comm_world.scatter
  (0, vsend.data(),twonums, vrecv.data(),twonums );
```

# Reduction and non-root processes

There are separate variants for root vs non-root processes in rooted collectives:

```
// scangather.cxx
const int root=0;
if (procno==root) {
  comm_world.reduce
    ( mpl::plus<int>(),root,
      my_number_of_elements,total_number_of_elements );
} else {
  comm_world.reduce
    ( mpl::plus<int>(),root,my_number_of_elements );
}
```

# Gather on non-root

Logically speaking, on every nonroot process, the gather call only has a send buffer. MPL supports this by having two variants that only specify the send data.

```
if (procno==root) {
  vector<int> size_buffer(nprocs);
  comm_world.gather
    (
     root,my_number_of_elements,size_buffer.data()
     );
} else {
  /*
  * If you are not the root, do versions with only send buffers
  */
  comm_world.gather
    ( root,my_number_of_elements );
```

# Reference: MPI_Gather

```
void mpl::communicator::gather
    ( int  root_rank, const T * senddata, const layout< T > &  sendl ) const
    ( int  root_rank, const T * senddata, const layout< T > &  sendl,
                              T * recvdata, const layout< T > &  recvl ) const
    // non-root versions:
    ( int  root_rank, const T & senddata ) const
    ( int  root_rank, const T & senddata, T *  recvdata ) const
```

# Reduce in place

The in-place variant is activated by specifying only one instead of two buffer arguments.

```
// separate recv buffer
comm_world.allreduce(mpl::plus<float>(), proc_data,reduce_data);
// in place
comm_world.allreduce(mpl::plus<float>(), proc_data);
```

Reducing a buffer requires specification of a *contiguous_layout*:

```
// collectbuffer.cxx
vector<float> rank2p2p1{ 2*xrank,2*xrank+1 },reduce2p2p1{0,0};
mpl::contiguous_layout<float> two_floats(rank2p2p1.size());
comm_world.allreduce
  (mpl::plus<float>(), rank2p2p1.data(),reduce2p2p1.data(),two_floats);
if ( iprint )
  cout << "Got: " << reduce2p2p1.at(0) << ","
         << reduce2p2p1.at(1) << endl;
```

Note that the buffers are of type *T* *, so it is necessary to take the *data*() of any *std*::*vector* and such.

# Layouts for gatherv

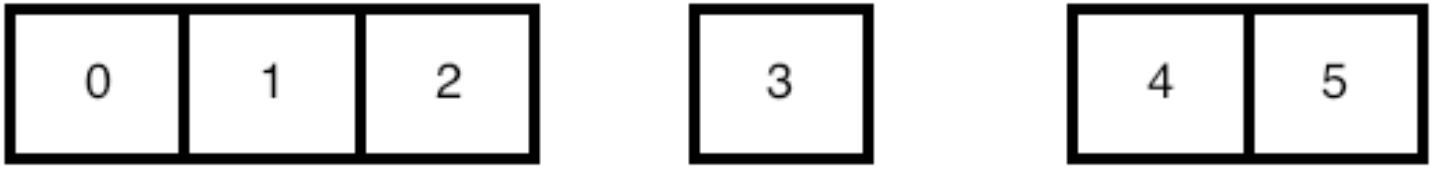
The size/displacement arrays for *MPI_Gatherv* / *MPI_Alltoallv* are handled through a *layouts* object, which is basically a vector of *layout* objects.

```
mpl::layouts<int> receive_layout;
for ( int iproc=0,loc=0; iproc<nprocs; iproc++ ) {
  auto siz = size_buffer.at(iproc);
  receive_layout.push_back
        ( mpl::indexed_layout<int>( {{ siz,loc }} ) );
  loc += siz;
}
```

# Scan operations

As in the C/F interfaces, MPL interfaces to the scan routines have the same calling sequences as the 'Allreduce' routine.

# For the next exercise



# Exercise 3 `scangather`

- Let each process compute a random value $n_{\mathrm{local}}$, and allocate an array of that length. Define

$$N = \sum n_{\mathrm{local}}$$

- Fill the array with consecutive integers, so that all local arrays, laid end-to-end, contain the numbers $0 \cdots N - 1$. (See figure 33.)

# Exercise 4 `scangather`

Take the code from exercise 3 and extend it to gather all local buffers onto rank zero. Since the local arrays are of differing lengths, this requires *`MPI_Gatherv`*.

How do you construct the lengths and displacements arrays?

# Operators

Arithmetic: *plus*, *multiplies*, *max*, *min*.

Logic: *logical_and*, *logical_or*, *logical_xor*.

Bitwise: *bit_and*, *bit_or*, *bit_xor*.

# User defined operators

A user-defined operator can be a templated class with an `operator()`. Example:

```
// reduceuser.cxx
template<typename T>
class lcm {
public:
  T operator()(T a, T b) {
    T zero=T();
    T t((a/gcd(a, b))*b);
    if (t<zero)
      return -t;
    return t;
  }
```

```
comm_world.reduce(lcm<int>(), 0, v, result);
```

(The templated class can be a lambda expression)

# Lambda reduction operators

You can also do the reduction by lambda:

MISSING SNIPPET mplreducecomplex in codesnippetsdir=../../snippets

# Nonblocking collectives

Nonblocking collectives have the same argument list as the corresponding blocking variant, except that instead of a *void* result, they return an *irequest*. (See 53)

Wait calls are methods of the *irequest* object.

```
// ireducescalar.cxx
float x{1.},sum;
auto reduce_request =
  comm_world.ireduce(mpl::plus<float>(), 0, x, sum);
reduce_request.wait();
if (comm_world.rank()==0) {
  std::cout << "sum = " << sum << '\n';
}
```

# 3. Point-to-point communication

# Blocking send and receive

In its scalar form, send and receive calls specify the data to be communicated and the source and target processes. MPL uses a default value for the tag, and it can deduce the type of the buffer. Sending a scalar becomes:

```
// sendscalar.cxx
if (comm_world.rank()==0) {
  double pi=3.14;
  comm_world.send(pi, 1);  // send to rank 1
  cout << "sent: " << pi << '\n';
} else if (comm_world.rank()==1) {
  double pi=0;
  comm_world.recv(pi, 0);  // receive from rank 0
  cout << "got : " << pi << '\n';
}
```

# Sending arrays

Sending vector data uses a pointer and a layout:

```
// sendbuffer.cxx
std::vector<double> v(8);
mpl::contiguous_layout<double> v_layout(v.size());
  comm_world.send(v.data(), v_layout, 1);  // send to rank 1
  comm_world.recv(v.data(), v_layout, 0);  // receive from rank 0
```

# Reference: MPI_Send

```
template<typename T >
void mpl::communicator::send
   ( const T scalar&,int dest,tag = tag(0) ) const
   ( const T *buffer,const layout< T > &,int dest,tag = tag(0) ) const
   ( iterT begin,iterT end,int dest,tag = tag(0) ) const
T : scalar type
begin : begin iterator
end : end iterator
```

# Reference: MPI_Recv

```
template<typename T >
status mpl::communicator::recv
   ( T &,int,tag = tag(0) ) const inline
   ( T *,const layout< T > &,int,tag = tag(0) ) const
   ( iterT  begin,iterT  end,int  source, tag  t = tag(0) ) const
```

# Exercise 5 `pingpongbuffer`

Implement the layout for a pingpong with an array.

# Proc null

Use `mpl::proc_null`.

# Message tag

MPL differs from other APIs in its treatment of tags: a tag is not directly an integer, but an object of class *tag_t*.

```
// sendrecv.cxx
mpl::tag_t t0(0);
comm_world.sendrecv
  ( mydata,sendto,t0,
    leftdata,recvfrom,t0 );
```

The *tag_t* class has a couple of methods such as *mpl::tag_t::any()* (for the *MPI_ANY_TAG* wildcard in receive calls) and *mpl::tag_t::up()* (maximal tag, found from the *MPI_TAG_UB* attribute).

# Tag types

Tag are *int* or an *enum* typ:

```
    template<typename T >
  tag_t (T t);
  tag_t (int t);

```

Example:

```
// inttag.cxx
enum class pingpongtag : int { ping=1, pong=2 };
int pinger = 0, ponger = world.size()-1;
if (world.rank()==pinger) {
  world.send(x, ponger, pingpongtag::ping);
  world.recv(x, ponger, pingpongtag::pong);
} else if (world.rank()==ponger) {
  world.recv(x, pinger, pingpongtag::ping);
  world.send(x, pinger, pingpongtag::pong);
}
```

# Any source

The constant `mpl::any_source` equals `MPI_ANY_SOURCE` (by `constexpr`).

# Send-recv call

The send-recv call in MPL has the same possibilities for specifying the send and receive buffer as the separate send and recv calls: scalar, layout, iterator. However, out of the nine conceivably possible routine signatures, only the versions are available where the send and receive buffer are specified the same way. Also, the send and receive tag need to be specified; they do not have default values.

```
// sendrecv.cxx
mpl::tag_t t0(0);
comm_world.sendrecv
  ( mydata,sendto,t0,
    leftdata,recvfrom,t0 );
```

```
// sendrecvarray.cxx
mpl::tag_t t0(0);
mpl::contiguous_layout<double> twofloats
      (2);
comm_world.sendrecv
  ( mydata,twofloats,sendto,t0,
    leftdata,twofloats,recvfrom,t0 );
```

# Status object

The *mpl::status_t* object is created by the receive (or wait) call:

```
mpl::contiguous_layout<double> target_layout(count);
mpl::status_t recv_status =
  comm_world.recv(target.data(),target_layout, the_other);
recv_count = recv_status.get_count<double>();
```

The *status* object can be queried:

```
int source = recv_status.source();
```

Likewise the source:

```
mpl::tag_t t = recv_status.tag();
```

# Receive count

The *get_count* function is a method of the status object. The argument type is handled through templating:

```
// recvstatus.cxx
double pi=0;
auto s = comm_world.recv(pi, 0);  // receive from rank 0
int c = s.get_count<double>();
std::cout << "got : " << c << " scalar(s): " << pi << '\n';
```

# Requests from nonblocking calls

Nonblocking routines have an *irequest* as function result. Note: not a parameter passed by reference, as in the C interface. The various wait calls are methods of the *irequest* class.

```
double recv_data;
mpl::irequest recv_request =
  comm_world.irecv( recv_data,sender );
recv_request.wait();
```

You can not default-construct the request variable:

```
// DOES NOT COMPILE:
mpl::irequest recv_request;
recv_request = comm.irecv( ... );
```

This means that the normal sequence of first declaring, and then filling in, the request variable is not possible.

*Implementation note: The wait call always returns a* `status_t` *object; not assigning it means that the destructor is called on it.*

# Request pools

Instead of an array of requests, use an *irequest_pool* object, which acts like a vector of requests, meaning that you can *push* onto it.

```
// irecvsource.cxx
mpl::irequest_pool recv_requests;
for (int p=0; p<nprocs-1; p++) {
  recv_requests.push( comm_world.irecv( recv_buffer[p], p ) );
}
```

You can not declare a pool of a fixed size and assign elements. (Why not? Can you find a way around it?)

# Wait any

The *irequest_pool* class has methods *waitany*, *waitall*, *testany*, *testall*, *waitsome*, *testsome*.

The 'any' methods return a *std::pair<mpl::test_result,size_t>*, where the *test_result* is an *enum class* with values:

- *completed* (for any/some/all completions),
- *no_completed* (for none),
- *no_active_requests* (if no more requests active).

```
auto [success,index] = recv_requests.waitany();
if ( success==mpl::test_result::completed ) {
      auto recv_status = recv_requests.get_status(index);
```

# Exercise 6 `setdiff`

Create two distributed arrays of positive integers. Take the set difference of the two: the first array needs to be transformed to remove from it those numbers that are in the second array.

How could you solve this with an *`MPI_Allgather`* call? Why is it not a good idea to do so? Solve this exercise instead with a circular bucket brigade algorithm.

# Buffered send

Creating and attaching a buffer is done through *bsend_buffer* and a support routine *bsend_size* helps in calculating the buffer size:

```
// bufring.cxx
vector<float> sbuf(BUFLEN), rbuf(BUFLEN);
int size{ comm_world.bsend_size<float>(mpl::contiguous_layout<float>(BUFLEN)) };
mpl::bsend_buffer buff(size);
comm_world.bsend(sbuf.data(),mpl::contiguous_layout<float>(BUFLEN), next);
```

Constant: *mpl::bsend_overhead* is *constexpr*'d to the MPI constant *MPI_BSEND_OVERHEAD*.

# Buffer attach and detach

There is a separate attach routine, but normally this is called by the constructor of the *bsend_buffer*. Likewise, the detach routine is called in the buffer destructor.

```
void mpl::environment::buffer_attach (void *buff, int size);
std::pair< void *, int > mpl::environment::buffer_detach ();
```

# Persistent requests

MPL returns a *prequest* from persistent 'init' routines, rather than an *irequest* (MPL note 53):

```
template<typename T >
prequest send_init (const T &data, int dest, tag t=tag(0)) const;
```

Likewise, there is a *prequest_pool* instead of an *irequest_pool* (note 54).

# 4. Derived Datatypes

# Datatype handling

MPL mostly handles datatypes through subclasses of the *layout* class. Layouts are MPL routines are templated over the data type.

```
// sendlong.cxx
mpl::contiguous_layout<long long> v_layout(v.size());
  comm.send(v.data(), v_layout, 1); // send to rank 1
```

Also works with complex of float and double.

The data types, where MPL can infer their internal representation, are enumeration types, C arrays of constant size and the template classes *std::array*, *std::pair* and *std::tuple* of the C++ Standard Template Library. The only limitation is, that the C array and the mentioned template classes hold data elements of types that can be sent or received by MPL.

# Native MPI datatypes

Should you need the *MPI_Datatype* object contained in an MPL layout, there is an access function *native_handle*.

# Large counts

Large buffer communication is supported:

```
// bigint.cxx
size_t s;
  vector<char> buffer(s);
  mpl::contiguous_layout<char> buffersize(s);
    comm_world.send( buffer.data(),buffersize,processB );
```

# Derived type handling

In MPL type creation routines are in the main namespace, templated over the datatypes.

```
// vector.cxx
vector<double>
  source(stride*count);
if (procno==sender) {
  mpl::strided_vector_layout<double>
    newvectortype(count,1,stride);
  comm_world.send
    (source.data(),newvectortype,the_other);
}
```

The commit call is part of the type creation, and freeing is done in the destructor.

# Contiguous type

The MPL interface makes extensive use of *contiguous_layout*, as it is the main way to declare a nonscalar buffer.

# Contiguous composing of types

Contiguous layouts can only use predefined types or other contiguous layouts as their 'old' type. To make a contiguous type for other layouts, use *vector_layout*:

```
// contiguous.cxx
mpl::contiguous_layout<int> type1(7);
mpl::vector_layout<int> type2(8,type1);
```

(Contrast this with *strided_vector_layout*; note 67.)

# Vector type

MPL has the *strided_vector_layout* class as equivalent of the vector type:

```
// vector.cxx
vector<double>
  source(stride*count);
if (procno==sender) {
  mpl::strided_vector_layout<double>
    newvectortype(count,1,stride);
  comm_world.send
    (source.data(),newvectortype,the_other);
}
```

(See note 66 for nonstrided vectors.)

# Iterator buffers

MPL point-to-point routines have a way of specifying the buffer(s) through a *begin* and *end* iterator.

```
// sendrange.cxx
vector<double> v(15);
  comm_world.send(v.begin(), v.end(), 1);  // send to rank 1
  comm_world.recv(v.begin(), v.end(), 0);  // receive from rank 0
```

Not available for collectives.

# Iterator layout

Noncontiguous iteratable objects can be send with a *iterator_layout*:

```
1 std::list<int> v(20, 0);
2 mpl::iterator_layout<int> l(v.begin(), v.end());
3 comm_world.recv(&(*v.begin()), l, 0);
```

# Subarray layout

The templated *subarray_layout* class is constructed from a vector of triplets of global size / subblock size / first coordinate.

```
mpl::subarray_layout<int>(
 { {ny, ny_1, ny_0}, {nx, nx_1, nx_0} }
);
```

# Indexed type

In MPL, the *indexed_layout* is based on a vector of 2-tuples denoting block length / block location.

```
// indexed.cxx
const int count = 5;
mpl::contiguous_layout<int>
  fiveints(count);
mpl::indexed_layout<int>
  indexed_where{ { {1,2}, {1,3}, {1,5}, {1,7}, {1,11} } };

if (procno==sender) {
  comm_world.send( source_buffer.data(),indexed_where, receiver );
} else if (procno==receiver) {
  auto recv_status =
    comm_world.recv( target_buffer.data(),fiveints, sender );
  int recv_count = recv_status.get_count<int>();
  assert(recv_count==count);
}
```

# Indexed block type

For the case where all block lengths are the same, use *indexed_block_layout*:

```
// indexedblock.cxx
mpl::indexed_block_layout<int>
  indexed_where(  1, {2,3,5,7,11} );
comm_world.send( source_buffer.data(),indexed_where, receiver );
```

# Struct type scalar

One could describe the MPI struct type as a collection of displacements, to be applied to any set of items that conforms to the specifications. An MPL *heterogeneous_layout* on the other hand, incorporates the actual data. Thus you could write

```
// structscalar.cxx
char c; double x; int i;
if (procno==sender) {
  c = 'x'; x = 2.4; i = 37; }
mpl::heterogeneous_layout object( c,x,i );
if (procno==sender)
  comm_world.send( mpl::absolute,object,receiver );
else if (procno==receiver)
  comm_world.recv( mpl::absolute,object,sender );
```

Here, the *absolute* indicates the lack of an implicit buffer: the layout is absolute rather than a relative description.

# Struct type general

More complicated data than scalars takes more work:

```
// struct.cxx
char c; vector<double> x(2); int i;
if (procno==sender) {
  c = 'x'; x[0] = 2.7; x[1] = 1.5; i = 37; }
mpl::heterogeneous_layout object
  ( c,
    mpl::make_absolute(x.data(),mpl::vector_layout<double>(2)),
    i );
if (procno==sender) {
  comm_world.send( mpl::absolute,object,receiver );
} else if (procno==receiver) {
  comm_world.recv( mpl::absolute,object,sender );
}
```

Note the *make_absolute* in addition to *absolute* mentioned above.

# Extent resizing

Resizing a datatype does not give a new type, but does the resize 'in place':

```
void layout::resize(ssize_t lb, ssize_t extent);
```

# 5. Communicator manipulations

# Communicator comparing

Code:

```
const mpl::communicator &comm =
  mpl::environment::comm_world();
MPI_Comm
  world_extract = comm.native_handle(),
  world_given = MPI_COMM_WORLD;
int result;
MPI_Comm_compare(world_extract,
      world_given,&result);
cout << "Compare raw comms: " << "\n"
     << "identical: "
     << (result==MPI_IDENT) << "\n"
     << "congruent: "
     << (result==MPI_CONGRUENT) << "\n"
     << "unequal  : "
     << (result==MPI_UNEQUAL) << "\n";
```

Output:

```
Compare raw comms:
identical: true
congruent: false
unequal  : false
```

# Communicator errhandler

MPL does not have a routine for setting the error handler. Instead, use the *native_handle* method to retrieve the embedded communiator.

# Communicator splitting

In MPL, splitting a communicator is done as one of the overloads of the communicator constructor;

```
// commsplit.cxx
// create sub communicator modulo 2
int color2 = procno % 2;
mpl::communicator comm2
  ( mpl::communicator::split, comm_world, color2 );
auto procno2 = comm2.rank();

// create sub communicator modulo 4 recursively
int color4 = procno2 % 2;
mpl::communicator
  comm4( mpl::communicator::split, comm2, color4 );
auto procno4 = comm4.rank();
```

*Implementation note: The `communicator::split` identifier is an object of class `communicator::split_tag`, itself is an otherwise empty subclass of `communicator`:*

```
class split_tag {};
static constexpr split_tag split{};
```

# Split by shared memory

Similar to ordinary communicator splitting (slide 79):

*communicator::split_shared*.

```
// commsplittype.cxx
mpl::communicator shared_comm
  ( mpl::communicator::split_shared_memory, world_comm );
int
  onnode_procno = shared_comm.rank(),
  onnode_nprocs = shared_comm.size();
```

But note: shared memory is currently not available, since windows are not (yet) implemented.

# 6. Process topologies

# Cartesian communicator

There is a separate class *cartesian_communicator*.

Additionaly, there is a class *dimensions* that describes the shape of the Cartesian grid and its periodicity. The *size(int)* method retrieves the size in the given dimension.

# Dims create

The *dims_create* routine takes a *dimensions* object with only the dimensionality specified, and creates one with the sizes filled in.

```
mpl::cartesian_communicator::dimensions brick(3);
brick = mpl::dims_create(nprocs,brick);
```

To have certain dimensions be periodic, the initial *dimensions* object needs to be create with periodicity values *periodic* or *non_periodic*.

```
mpl::cartesian_communicator::dimensions pbrick
  ( { mpl::cartesian_communicator::non_periodic,
      mpl::cartesian_communicator::periodic,
      mpl::cartesian_communicator::non_periodic } );
pbrick = mpl::dims_create(nprocs,pbrick);
```

# Cartesian communicator create

The actual Cartesian communicator has a constructor that takes a *dimensions* object as input.

```
mpl::cartesian_communicator cart_comm( comm_world,brick );
```

# Get the dimensions object

The *dimensions* object can be extracted from the communicator

```
mpl::cartesian_communicator::dimensions
  dimensions = cart_comm.get_dimensions();
```

after which dimensions and periodicies can be extracted:

```
// cartcoord.cxx
int dsize = dimensions.size(idim);
auto p = dimensions.periodicity(idim);
```

# Rank to coord translation

The *coordinates* method of the cartesian communicator returns a vector-like object describing the process coordinate:

```
for ( int ip=0; ip<nprocs; ip++ ) {
  mpl::cartesian_communicator::vector
        coord = cart_comm.coordinates(ip);
  print("[{:2}] coord: [",ip);
  for ( int id=0; id<dim; id++ )
        print("{},",coord[id]);
  print("]\n");
```

# Cartesian shifting

The routine *cartesian_communicator::shift* takes a dimension and a direction, and gives the source and destination as a *shifted_ranks* object, which is basically a tuple of two integers:

```
int pred,succ;
mpl::shift_ranks shifted = cart_comm.shift
        ( /* dim: */ 1,/* up: */ 1 );
pred = shifted.source; succ = shifted.destination;
```

# Graph communicators

The constructor *dist_graph_communicator*

```
dist_graph_communicator
   (const communicator &old_comm, const source_set &ss,
    const dest_set &ds, bool reorder = true);
```

is a wrapper around *MPI_Dist_graph_create_adjacent*.

# Graph communicator querying

Methods *indegree*, *outdegree* are wrappers around *MPI_Dist_graph_neighbors_count*. Sources and targets can be queried with *inneighbors* and *outneighbors*, which are wrappers around *MPI_Dist_graph_neighbors*.

# 7. Other

# Timing

The timing routines *wtime* and *wtick* and *wtime_is_global* are environment methods:

```
double  mpl::environment::wtime ();
double  mpl::environment::wtick ();
bool mpl::environment::wtime_is_global ();
```

# Info objects

There is an *info* object in the *mpl* namespace:

```
mpl::info infoobject; // default constructor
```

Sample methods:

```
void set(const std::string &key, const std::string &value);
[[nodiscard]] std::optional<std::string> value(const std::string &key) const;
```

# Threading support

MPL always calls *MPI_Init_thread* requesting the highest level *MPI_THREAD_MULTIPLE*.

```
enum mpl::threading_modes {
  mpl::threading_modes::single = MPI_THREAD_SINGLE,
  mpl::threading_modes::funneled = MPI_THREAD_FUNNELED,
  mpl::threading_modes::serialized = MPI_THREAD_SERIALIZED,
  mpl::threading_modes::multiple = MPI_THREAD_MULTIPLE
};
threading_modes mpl::environment::threading_mode ();
bool mpl::environment::is_thread_main ();
```

# Missing from MPL

MPL is not a full MPI implementation

- One-sided communication
- Shared memory
- Process management